Quaderno AI

Concetti, strumenti e percorsi pratici per lo sviluppo software con AI

Andrea Merlin

Numero 1 - Novembre 2026

Indice

Capitolo 2 **1**

Un MCP Server con Entity Framework Core: dati, query e primitivi avanzati . . . 1
2.1 Perché un database in un MCP server . . . 2
2.2 Entity Framework Core in sintesi . . . 3
2.3 Il dominio di esempio: un catalogo prodotti . . . 6
2.4 Setup del progetto e pacchetti . . . 8
2.5 Entità e DbContext . . . 10
2.6 Connessione e creazione del database . . . 12
2.7 Le migrazioni . . . 14
2.8 I tool di lettura . . . 15
2.9 I tool di scrittura . . . 18
2.10 Il pattern IDbContextFactory . . . 21
2.11 Il server stdio completo . . . 23
2.12 Il server HTTP completo . . . 28
2.13 Testare il server con l'MCP Inspector . . . 31
2.14 Primitivi avanzati: il Progress . . . 32
2.15 Riepilogo, glossario ed esercizi . . . 34

Capitolo 2

Un MCP Server con Entity Framework Core: dati, query e primitivi avanzati

Abstract. Nel primo capitolo abbiamo costruito server MCP che espongono tool "senza memoria": file system e chiamate HTTP. Ora facciamo il passo successivo: un server che parla con un **database relazionale** tramite **Entity Framework Core**, il framework ORM di .NET. Implementeremo un catalogo prodotti completo — letture, ricerche, inserimenti, aggiornamenti e cancellazioni — scopriremo il pattern giusto per usare un `DbContext` dentro un tool, affronteremo le migrazioni e i trasporti `stdio` e HTTP, e infine vedremo come un tool può comunicare il proprio avanzamento con il primitivo **Progress**.

Cosa imparerai in questo capitolo

Al termine della lettura e della pratica saprai:

- spiegare perché un agente ha bisogno di **dati persistenti** e come un MCP server può fornirglieli;
- usare **Entity Framework Core 10** per modellare entità e relazioni;
- collegare un `DbContext` a un tool MCP con il pattern `IDbContextFactory`, evitando i problemi di lifetime;
- creare il database con `EnsureCreated` e, per i progetti reali, con le **migrazioni** (`dotnet ef`);
- implementare tool di **lettura** e **scrittura** con query LINQ, filtri e gestione degli errori;
- esporre lo stesso server su `stdio` e su **HTTP**;
- usare il primitivo **Progress** per comunicare l'avanzamento di un tool.

Prerequisiti

- Aver letto (o padroneggiare) il Capitolo 1: in particolare tool, trasporti, configurazione dei client e MCP Inspector;
- .NET SDK 10 e un editor (VS Code, Visual Studio o Rider);

- conoscenza di base del linguaggio **LINQ** e, facoltativamente, di SQL.

Convenzioni

Valgono le convenzioni del Capitolo 1. In più, in questo capitolo useremo **SQLite** come database: è un database relazionale completo, basato su file, che non richiede alcun server da installare — perfetto per un server `stdio` locale. I concetti e il codice si trasferiscono quasi identici a SQL Server o PostgreSQL, cambiando solo il provider (lo vedremo).

Come nel capitolo precedente, ogni frammento è stato **compilato ed eseguito** con .NET 10, `ModelContextProtocol` 2.3.0 ed Entity Framework Core 10.0.14.

2.1 Perché un database in un MCP server

Nel Capitolo 1 i nostri server erano "senza stato": il server del file system leggeva e scriveva file, quello meteo interrogava un'API esterna, ma nessuno **ricordava** nulla tra una chiamata e l'altra. In moltissimi scenari reali, però, l'agente ha bisogno di dati che **persistono**, si **interrogano** e si **aggregano**.

2.1.1 Cosa chiede un agente a un database

Pensa a un assistente aziendale. Le sue richieste tipiche sono:

- "Quanti ordini sono stati evasi la settimana scorsa?" — una **aggregazione**;
- "Trovami i clienti che non comprano da più di sei mesi" — una **query con filtro**;
- "Aggiungi questo prodotto al catalogo con prezzo 29,90" — una **scrittura**;
- "Quali sono le categorie più vendute?" — una **join** con raggruppamento.

Nessuna di queste operazioni è ragionevolmente realizzabile leggendo file di testo o chiamando API generiche: serve un **motore di interrogazione**, e un database relazionale è lo strumento naturale.

2.1.2 Il ruolo del MCP server

Il MCP server si colloca esattamente nel punto giusto: è il **ponte** tra il modello e il database. Da un lato espone *tool* descrittivi (`list_products`, `create_product`, …) che il modello scopre e invoca; dall'altro traduce quei tool in **query SQL** (o, meglio, in query LINQ che il provider traduce in SQL) e restituisce risultati strutturati.

Il modello non sa nulla di tabelle, indici o join: vede un menu di strumenti ben descritti. Il database non sa nulla di modelli o di prompt: riceve query. Il server è il traduttore che vive in mezzo, e il suo compito è rendere l'incontro **sicuro, controllato e coerente**.

2.1.3 Perché Entity Framework Core

Potremmo usare ADO.NET o un micro-ORM come Dapper per scrivere SQL a mano. Ma Entity Framework Core offre tre vantaggi decisivi per questo scenario:

1. **Modello a oggetti.** Definiamo entità C# (`Product`, `Category`) e relazioni; EF Core si occupa dello schema e delle query.
2. **LINQ.** Le query si scrivono in C# tipizzato, con il completamento automatico e il controllo dei tipi a compile time.
3. **Provider multipli.** Lo stesso codice funziona con SQLite, SQL Server, PostgreSQL, MySQL e altri, cambiando un solo pacchetto e una stringa di connessione.

In più, EF Core è il modo **idiomatico** di accedere ai dati in .NET: un lettore che conosce ASP.NET Core troverà naturale integrare un `DbContext` in un server MCP.

2.1.4 La sfida specifica di MCP: il lifetime del DbContext

C'è una complicazione, ed è il motivo per cui questo capitolo esiste: un `DbContext` è progettato per vivere **poco**, legato a una singola unità di lavoro. In una richiesta web, per esempio, vive per la durata della richiesta. In un server MCP, invece, ogni invocazione di tool è un "lavoro" a sé, e il server resta in vita a lungo.

Se iniettassimo un singolo `DbContext` condiviso tra tutte le invocazioni, incapperemmo in problemi classici: contesto non thread-safe, tracking che cresce senza controllo, dati stantii. La soluzione corretta — che esploreremo e useremo — è il `IDbContextFactory<T>`: una factory che crea un contesto nuovo per ogni invocazione. Dedichiamo a questo pattern un'intera sezione, perché è il cuore dell'integrazione tra EF Core e MCP.

2.1.5 La mappa del capitolo

Partiremo da un ripasso essenziale di EF Core, poi definiremo il dominio di esempio (un catalogo prodotti), creeremo il progetto, i modelli e il database. Implementeremo i tool di lettura e scrittura, li esporremo su `stdio` e HTTP, e infine vedremo come comunicare il progresso di un'operazione. Alla fine avrai un server MCP **con persistenza** completo, pronto per essere collegato a Claude Desktop o VS Code.

2.2 Entity Framework Core in sintesi

Prima di scrivere codice, ripassiamo i concetti essenziali di Entity Framework Core. Se lo conosci già, puoi scorrere velocemente: serve solo ad allineare il vocabolario.

2.2.1 Cos'è un ORM

Entity Framework Core è un **Object-Relational Mapper** (ORM): una libreria che mappa le **entità** C# (classi) sulle **tabelle** del database e le **proprietà** sulle **colonne**. Il programmatore lavora con oggetti e LINQ; l'ORM traduce il tutto in SQL e materializza i risultati in oggetti.

Il punto di ingresso è il `DbContext`: una classe che rappresenta una sessione di lavoro con il database. Contiene un `DbSet<T>` per ogni entità e permette di eseguire query, inserimenti, aggiornamenti e cancellazioni, coordinati dal metodo `SaveChanges()`.

2.2.2 Le entità

Un'entità è una semplice classe C# con proprietà pubbliche. Per convenzione, la proprietà `Id` (o `<NomeClasse>Id`) viene riconosciuta come **chiave primaria**:

```
public sealed class Category
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public List<Product> Products { get; set; } = [];
}
```

La proprietà `Products` è una **navigation property**: rappresenta la relazione "una categoria ha molti prodotti". EF Core la userà per eseguire le join quando serve.

2.2.3 Il DbContext

La classe `DbContext` dichiara quali entità fanno parte del modello:

```
public sealed class CatalogDbContext(DbContextOptions<CatalogDbContext>
 ↪  options)
    : DbContext(options)
{
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();
}
```

Il costruttore primario riceve `DbContextOptions<T>`, che contiene la configurazione (provider, stringa di connessione) e che verrà fornita dal sistema di dependency injection.

2.2.4 Le query LINQ

EF Core traduce le query LINQ in SQL. Un esempio:

```
var books = await db.Products
    .Where(p => p.Category!.Name == "Books")
```

```
    .OrderBy(p => p.Name)
    .ToListAsync();
```

L'esecuzione avviene **solo quando si materializza la query** (con `ToListAsync`, `ToArrayAsync`, `FirstOrDefaultAsync`, ecc.): fino a quel momento si compone un albero di espressioni che il provider traduce. Questo dettaglio è importante perché ci permette di costruire query **condizionali**, aggiungendo filtri solo se necessari.

2.2.5 Le operazioni di scrittura

Le scritture passano dal *change tracker* di EF Core:

```
db.Products.Add(new Product { Name = "Headphones", Price = 199.99m,
↪  CategoryId = 1 });
await db.SaveChangesAsync(); // genera INSERT
```

`SaveChanges()` rileva le modifiche (aggiunte, modifiche, cancellazioni) e le applica al database in una **transazione implicita**.

2.2.6 I provider

EF Core separa il nucleo dai **provider**, uno per database. I più usati:

- **SQLite:** `Microsoft.EntityFrameworkCore.Sqlite`, adatto allo sviluppo locale e a database su file.
- **SQL Server:** `Microsoft.EntityFrameworkCore.SqlServer`, usato spesso in ambienti Windows e Azure.
- **PostgreSQL:** `Npgsql.EntityFrameworkCore.PostgreSQL`, provider open source.
- **InMemory:** `Microsoft.EntityFrameworkCore.InMemory`, utile per alcuni test ma non equivalente a un vero database relazionale.

In questo capitolo useremo **SQLite** per la sua semplicità operativa: zero installazioni, un solo file. Alla fine del capitolo vedremo come passare a un altro provider cambiando poco più di una riga.

2.2.7 La configurazione tramite DI

In un'applicazione .NET moderna, il `DbContext` si registra nel container di dependency injection. EF Core offre due modalità:

- `AddDbContext<T>` — registra un contesto **scoped** (condiviso nell'ambito di una richiesta);
- `AddDbContextFactory<T>` — registra una **factory** che crea un contesto nuovo a richiesta.

Come anticipato, per i tool MCP useremo `AddDbContextFactory<T>`: ogni invocazione di tool avrà il suo contesto fresco, senza conflitti e senza tracking accumulato. Ne vedremo i dettagli operativi nella sezione dedicata.

2.3 Il dominio di esempio: un catalogo prodotti

Per tutto il capitolo lavoreremo su un dominio semplice ma realistico: un **catalogo prodotti** organizzato in **categorie**. Due entità, una relazione, e un insieme di operazioni che coprono tutti i casi tipici: lettura con filtro, creazione, cancellazione.

2.3.1 Il modello concettuale

- **Category** — una categoria di prodotti (per esempio "Books", "Electronics"). Ha un nome univoco.
- **Product** — un prodotto che appartiene a **una** categoria. Ha nome, prezzo e un riferimento alla categoria.

La relazione è **uno-a-molti**: una categoria ha molti prodotti, ogni prodotto ha una sola categoria.



Figura 1: Relazione uno-a-molti tra `Category` e `Product`

2.3.2 Le entità C#

Traduciamo il modello in classi. Nota l'uso di `required` (C# 11+) per rendere obbligatorie le proprietà non nullable, e della collection expression `[]` per inizializzare la navigation:

```
public sealed class Category
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public List<Product> Products { get; set; } = [];
}

public sealed class Product
{
    public int Id { get; set; }
```

```
    public required string Name { get; set; }
    public decimal Price { get; set; }
    public int CategoryId { get; set; }
    public Category? Category { get; set; }
}
```

Alcune osservazioni:

- `Id` è la chiave primaria per convenzione.
- `Product.CategoryId` è la **chiave esterna**: EF Core la riconosce per convenzione (nome `Category` + `Id`) e la userà per la relazione.
- `Product.Category` è la navigation verso il "lato uno"; `Category.Products` verso il "lato molti". Entrambe servono per scrivere query leggibili con `Include`.

2.3.3 Cosa esporremo come tool

Dal modello deriviamo un set di tool MCP sensato:

Tool	Tipo	Descrizione
`list_products`	Lettura	Elenca i prodotti, con filtro opzionale per categoria
`search_products`	Lettura	Cerca prodotti per nome (sottostringa)
`get_product`	Lettura	Restituisce un singolo prodotto per id
`create_product`	Scrittura	Crea un prodotto in una categoria
`update_price`	Scrittura	Aggiorna il prezzo di un prodotto
`delete_product`	Scrittura (distruttiva)	Elimina un prodotto per id

A questi aggiungeremo, nella sezione sul Progress, un tool dimostrativo per comunicare l'avanzamento di un'operazione.

2.3.4 Il tipo di ritorno: un DTO

Per le letture non restituiremo le entità EF Core "grezze": le proietteremo in un **DTO** (Data Transfer Object) con LINQ `Select`. Così controlliamo esattamente cosa viaggia verso il modello, evitando di serializzare navigation property potenzialmente cicliche (una categoria che contiene i suoi prodotti, che contengono la categoria…) e dati non necessari:

```
public sealed record ProductDto(int Id, string Name, decimal Price,
↪  string Category);
```

Il DTO è un `record` immutabile: pulito, serializzabile e facile da leggere. Il nome della categoria viene "appiattito" nel DTO, così il modello riceve già il dato pronto.

2.3.5 Il principio guida del capitolo

Tutto il codice che segue ruota attorno a un'idea: **il database resta nascosto dietro i tool**. Il modello non sa che c'è una tabella `Products`; sa che c'è un tool `list_products` con un parametro `category`. Il server decide quali query eseguire, come validare gli input e quali errori restituire. Questa separazione è ciò che rende l'integrazione sicura e manutenibile.

2.4 Setup del progetto e pacchetti

Prepariamo l'ambiente. Il procedimento ricalca il Capitolo 1, con l'aggiunta dei pacchetti EF Core.

2.4.1 Creare il progetto

Come per ogni server `stdio`, partiamo da un progetto console:

```
> mkdir CatalogMcpServer
> cd CatalogMcpServer
> dotnet new console -n CatalogMcpServer -o .
```

2.4.2 Aggiungere i pacchetti

Servono quattro pacchetti:

```
> dotnet add package ModelContextProtocol --version 2.3.0
> dotnet add package Microsoft.Extensions.Hosting
> dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version
 ↪  10.0.14
> dotnet add package Microsoft.EntityFrameworkCore.Design --version
 ↪  10.0.14
```

A cosa serve ciascuno:

- `ModelContextProtocol` — il C# SDK di MCP (con le estensioni DI);
- `Microsoft.Extensions.Hosting` — l'host generico (`Host.CreateApplicationBuilder`) per il server `stdio`;
- `Microsoft.EntityFrameworkCore.Sqlite` — il provider EF Core per SQLite;
- `Microsoft.EntityFrameworkCore.Design` — gli strumenti di design-time, necessari per il comando `dotnet ef` (migrazioni). Si usa solo in sviluppo: per questo, se preferisci, puoi marcarlo con `PrivateAssets="all"`, ma non è indispensabile.

2.4.3 Il file di progetto risultante

Dopo l'aggiunta, il file `CatalogMcpServer.csproj` assomiglierà a:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design"
↪   Version="10.0.14" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite"
↪   Version="10.0.14" />
    <PackageReference Include="Microsoft.Extensions.Hosting"
↪   Version="10.0.13" />
    <PackageReference Include="ModelContextProtocol" Version="2.3.0" />
  </ItemGroup>

</Project>
```

> **Nota sulle versioni.** `Microsoft.Extensions.Hosting` compare con la versione che `dotnet add package` risolve automaticamente (qui 10.0.13); non serve fissarla a mano, purché sia una 10.x. Per l'SDK MCP e per EF Core, invece, fissiamo le versioni indicate per riproducibilità.

2.4.4 Installare gli strumenti EF Core (una tantum)

Il comando `dotnet ef` (che useremo per le migrazioni) non fa parte dell'SDK .NET di base: va installato come strumento globale, una sola volta per macchina:

```
> dotnet tool install --global dotnet-ef
```

Se è già installato, puoi aggiornarlo con:

```
> dotnet tool update --global dotnet-ef
```

Verifica che sia disponibile:

```
> dotnet ef --version
```

2.4.5 Verificare la compilazione

Prima di scrivere una riga di codice nostro, assicuriamoci che il progetto compili:

```
> dotnet build
```

Con l'ambiente pronto, passiamo a definire il modello e il `DbContext`.

2.5 Entità e DbContext

Creiamo ora le entità e il contesto. Per semplicità in questo capitolo teniamo tutto in `Program.cs` (top-level statements + classi); in un progetto reale separeresti le classi in file dedicati, ma per un esempio didattico la vicinanza del codice aiuta la lettura.

2.5.1 Le entità

```
public sealed class Category
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public List<Product> Products { get; set; } = [];
}

public sealed class Product
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public decimal Price { get; set; }
    public int CategoryId { get; set; }
    public Category? Category { get; set; }
}
```

2.5.2 Il DbContext

```
using Microsoft.EntityFrameworkCore;

public sealed class CatalogDbContext(DbContextOptions<CatalogDbContext>
 ↪  options)
    : DbContext(options)
{
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Product>()
            .Property(p => p.Price)
            .HasPrecision(18, 2);
    }
```

```
}
```

2.5.3 La configurazione del modello

Nel metodo `OnModelCreating` dichiariamo la **precisione** del prezzo: `HasPrecision(18, 2)` significa 18 cifre totali di cui 2 decimali. È una buona pratica per le colonne monetarie: evita che SQLite (che di default non applica vincoli di precisione ai `decimal`) tronchi o approssimi i valori in modo imprevisto.

Gran parte della configurazione non serve scriverla: EF Core riconosce per convenzione:

- la chiave primaria (`Id`);
- la chiave esterna (`CategoryId`);
- la relazione uno-a-molti tramite le navigation property.

2.5.4 Un vincolo utile: il nome categoria univoco

Per rendere il modello più robusto, aggiungiamo un **indice univoco** sul nome della categoria, così il database stesso rifiuterà i duplicati:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Property(p => p.Price)
        .HasPrecision(18, 2);

    modelBuilder.Entity<Category>()
        .HasIndex(c => c.Name)
        .IsUnique();
}
```

Con questo vincolo, se un tool tentasse di inserire due categorie con lo stesso nome, il database risponderebbe con un errore di vincolo, che noi tradurremo in un messaggio chiaro per il modello.

2.5.5 Perché `required` e nullable

Le proprietà `Name` sono `required string` (non nullable): dichiarano un vincolo a compile time che aiuta a evitare oggetti "mezzi vuoti". Le navigation `Category?` e la collection `Products` sono invece gestite da EF Core: la collection parte da una lista vuota, la navigation verso il "lato uno" può essere `null` finché non viene caricata. Con i **nullable reference types** abilitati (come nel template), questo codice comunica con precisione cosa può essere `null` e cosa no.

2.5.6 Il prossimo passo

Con entità e contesto pronti, dobbiamo decidere **come** creare il database fisico. Le opzioni sono due — `EnsureCreated` (rapido, per prototipi) e le **migrazioni** (per progetti reali) — e le vedremo nelle prossime due sezioni.

2.6 Connessione e creazione del database

Colleghiamo il `DbContext` all'applicazione e creiamo il database. Useremo due approcci: `EnsureCreated` per partire subito, e le **migrazioni** (sezione successiva) per il flusso di lavoro professionale.

2.6.1 La stringa di connessione

Per SQLite la stringa di connessione è semplicissima: un percorso di file.

```
builder.Services.AddDbContextFactory<CatalogDbContext>(options =>
    options.UseSqlite("Data Source=catalog.db"));
```

`Data Source=catalog.db` indica il file del database, relativo alla cartella di lavoro del processo. Per un server `stdio` lanciato dal client, la cartella di lavoro è quella dichiarata nella configurazione del client; per uno lanciato a mano, la cartella corrente.

2.6.2 AddDbContextFactory

Nota che usiamo **`AddDbContextFactory`**, non `AddDbContext`. La differenza:

- `AddDbContext<CatalogDbContext>` registra un contesto **scoped**: pensato per essere condiviso durante una singola richiesta web, con un solo `SaveChanges` finale.
- `AddDbContextFactory<CatalogDbContext>` registra una **`IDbContextFactory<CatalogDbContext>`**: una factory che crea un contesto nuovo, indipendente, ogni volta che serve.

Come vedremo nella sezione dedicata, il factory è la scelta giusta per i tool MCP, perché ogni invocazione di tool è un'unità di lavoro isolata.

2.6.3 Creare lo schema con EnsureCreated

Per i prototipi, il modo più rapido per avere un database funzionante è `EnsureCreated`:

```
var options = new DbContextOptionsBuilder<CatalogDbContext>()
    .UseSqlite("Data Source=catalog.db")
    .Options;
await using var db = new CatalogDbContext(options);
await db.Database.EnsureCreatedAsync();
```

`EnsureCreated` crea il database e lo schema **se non esistono**. Non usare `EnsureDeleted` all'avvio: elimina anche i dati e i comandi EF Core possono eseguire il codice di avvio in fase di design. Attenzione: `EnsureCreated` non si integra con le migrazioni — se il modello cambia, non aggiorna lo schema esistente. È uno strumento da prototipo.

2.6.4 Seed dei dati di esempio

Il blocco seguente è una **continuazione** del precedente: va eseguito nello stesso scope, mentre `db` è ancora aperto. Per avere qualcosa da interrogare, inseriamo due categorie e due prodotti:

```
if (!await db.Products.AnyAsync())
{
    var books = await db.Categories.SingleOrDefaultAsync(c => c.Name ==
     ↪  "Books")
        ?? new Category { Name = "Books" };
    var electronics = await db.Categories.SingleOrDefaultAsync(c =>
     ↪  c.Name == "Electronics")
        ?? new Category { Name = "Electronics" };

    db.Products.AddRange(
        new Product { Name = "The Lord of the Rings", Price = 29.90m,
         ↪  Category = books },
        new Product { Name = "MacBook Air", Price = 1199.00m, Category
         ↪  = electronics });

    await db.SaveChangesAsync();
}
```

Nota l'uso delle **navigation property** (`Category = books`) invece della chiave esterna: EF Core risolve da solo il valore di `CategoryId` quando salva.

2.6.5 Dove mettere questa inizializzazione

Per un server `stdio`, conviene eseguire l'inizializzazione **prima** di avviare il server, così il database è pronto quando arrivano le prime richieste. Nel codice completo del server (sezione 2.11) vedremo il blocco di inizializzazione piazzato tra la configurazione dei servizi e `RunAsync()`.

Per creare il `DbContext` in quel punto abbiamo bisogno delle opzioni: le costruiamo a mano con `DbContextOptionsBuilder`, oppure risolviamo la factory dal container. Entrambe le strade sono valide; per l'esempio useremo la prima, più esplicita.

2.7 Le migrazioni

`EnsureCreated` va bene per un prototipo, ma un progetto reale evolve: il modello cambia (nuove colonne, nuove tabelle) e il database deve seguirlo **senza perdere i dati**. Le **migrazioni** di EF Core risolvono esattamente questo problema: registrano le modifiche allo schema come una sequenza di passi versionati, applicabili e reversibili.

2.7.1 Come funzionano

Una migrazione è un file C# che descrive una **transizione** dello schema (per esempio "aggiungi la colonna `Stock` alla tabella `Products`"). Il comando `dotnet ef` confronta il modello attuale con l'ultima migrazione e genera la migrazione successiva. Applicando le migrazioni in ordine, il database raggiunge lo stato desiderato.

2.7.2 Creare la prima migrazione

Con il modello definito, generiamo la migrazione iniziale:

```
> dotnet ef migrations add InitialCreate
```

Il comando crea una cartella `Migrations/` con due file: la migrazione (con i metodi `Up` e `Down`) e uno snapshot del modello. `InitialCreate` è il nome che diamo alla migrazione; scegli nomi descrittivi ("AddStockColumn", "AddCustomerTable").

2.7.3 Applicare la migrazione

Per creare (o aggiornare) il database:

```
> dotnet ef database update
```

Questo comando applica tutte le migrazioni pendenti. Al termine, il database è allineato al modello.

2.7.4 Il ciclo di lavoro quotidiano

Il flusso tipico è:

1. modifichi il modello (aggiungi una proprietà, una tabella);
2. generi una migrazione: `dotnet ef migrations add <NomeDescrittivo>`;
3. applichi: `dotnet ef database update`;
4. versioni la cartella `Migrations/` insieme al codice.

Mai modificare a mano una migrazione già applicata in produzione: meglio aggiungerne una nuova.

2.7.5 Migrazioni vs EnsureCreated

Aspetto	`EnsureCreated`	Migrazioni
Scopo	Prototipi, test	Produzione
Aggiorna lo schema esistente?	No	Sì, incrementalmente
Conserva i dati?	Non rilevante (ricrea)	Sì
Versionabile?	No	Sì (cartella `Migrations/`)
Comando	Codice	`dotnet ef`

Per il resto del capitolo useremo `EnsureCreated` per la semplicità dell'esempio, ma in produzione scegli le migrazioni: il tuo database (e chi lo usa) ti ringrazierà.

2.7.6 Migrazioni e server MCP

Un dettaglio operativo: i comandi `dotnet ef` hanno bisogno di poter **creare un'istanza** del `DbContext` in fase di design. Quando il `DbContext` è registrato via `AddDbContextFactory` e il progetto è un eseguibile, `dotnet ef` di solito riesce a istanziarlo da solo; se così non fosse (perché il costruttore richiede opzioni non ovvie), si può aggiungere un costruttore senza parametri che configura SQLite di default, oppure un `IDesignTimeDbContextFactory`. Per il nostro progetto semplice non serve: la stringa di connessione è già nel codice.

2.8 I tool di lettura

Veniamo al cuore del capitolo: i tool. Iniziamo dalle **letture**, che sono le operazioni più frequenti e quelle in cui LINQ dà il meglio.

2.8.1 La classe dei tool

La classe riceve la factory nel costruttore e la usa per creare un contesto per ogni invocazione:

```
using Microsoft.EntityFrameworkCore;
using ModelContextProtocol.Server;
using System.ComponentModel;

[McpServerToolType]
public sealed class CatalogTool(IDbContextFactory<CatalogDbContext>
 ↪  factory)
{
    // I tool possono usare direttamente il parametro del costruttore
     ↪  primario.
}
```

Il costruttore primario riceve la factory; i parametri del costruttore primario sono accessibili direttamente nei metodi della classe (come abbiamo già visto con `HttpClient` nel Capitolo 1), quindi non serve dichiarare campi aggiuntivi.

2.8.2 Elencare i prodotti

```
[McpServerTool(Name = "list_products"), Description("Lists all products,
 ↪  optionally filtered by category name.")]
public async Task<ProductDto[]> ListProducts(
    [Description("Optional category name filter.")] string? category =
     ↪  null,
    CancellationToken cancellationToken = default)
{
    await using var db = await
 ↪  factory.CreateDbContextAsync(cancellationToken);

    IQueryable<Product> query = db.Products;

    if (category is not null)
        query = query.Where(p => p.Category!.Name == category);

    return await query
        .OrderBy(p => p.Name)
        .Select(p => new ProductDto(p.Id, p.Name, p.Price,
         ↪  p.Category!.Name))
        .ToArrayAsync(cancellationToken);
}
```

Analizziamolo:

- **`CreateDbContextAsync`** crea un contesto nuovo; **`await using`** ne garantisce la corretta chiusura (rilascia la connessione) a fine metodo.
- Con la proiezione in **`Select`**, **`Include(p => p.Category)`** è superfluo: EF Core traduce l'accesso a **`p.Category.Name`** nella query SQL della proiezione. È la proiezione, non **`Include`**, a evitare query aggiuntive.
- **Filtro condizionale**: se **`category`** è **`null`**, la clausola **`Where`** non viene aggiunta; altrimenti filtriamo per nome. La query è composta prima, eseguita dopo.
- **Proiezione con `Select`**: invece delle entità restituiamo un **`ProductDto`** per riga, con il nome della categoria "appiattito".
- **`ToArrayAsync`** materializza la query (la esegue) e restituisce l'array; **`cancellationToken`** annulla sia la query sia la creazione del contesto.

2.8.3 Cercare per nome

```
[McpServerTool(Name = "search_products"), Description("Searches products
 ↪  whose name contains the given text.")]
```

```
public async Task<ProductDto[]> SearchProducts(
    [Description("Text to search in product names.")] string term,
    CancellationToken cancellationToken = default)
{
    await using var db = await
    ↪ factory.CreateDbContextAsync(cancellationToken);

    return await db.Products
        .Where(p => p.Name.Contains(term))
        .OrderBy(p => p.Name)
        .Select(p => new ProductDto(p.Id, p.Name, p.Price,
        ↪  p.Category!.Name))
        .ToArrayAsync(cancellationToken);
}
```

`Contains` viene tradotto in un `LIKE '%term%'` (con SQLite è case-insensitive per i caratteri ASCII): utile per la ricerca "fuzzy" che un modello spesso richiede.

2.8.4 Ottenere un singolo prodotto

```
[McpServerTool(Name = "get_product"), Description("Gets a single product
↪  by id.")]
public async Task<ProductDto?> GetProduct(
    [Description("Product id.")] int id,
    CancellationToken cancellationToken = default)
{
    await using var db = await
    ↪ factory.CreateDbContextAsync(cancellationToken);

    return await db.Products
        .Where(p => p.Id == id)
        .Select(p => new ProductDto(p.Id, p.Name, p.Price,
        ↪  p.Category!.Name))
        .FirstOrDefaultAsync(cancellationToken);
}
```

Restituiamo `ProductDto?`: se il prodotto non esiste, il risultato è `null`, e il modello potrà dire all'utente "non ho trovato il prodotto 42". In alternativa si può lanciare una `McpException` (come vedremo per le scritture); per le letture, un `null` è spesso la scelta più naturale.

2.8.5 Schema generato

Come visto nel Capitolo 1, l'SDK genera lo schema di input dai parametri. Per `ListProducts` produrrà, in sostanza:

```
{
  "type": "object",
```

```
  "properties": {
    "category": {
      "description": "Optional category name filter.",
      "type": ["string", "null"],
      "default": null
    }
  }
}
```

Il parametro nullable con default diventa opzionale e può essere `null`: il modello potrà chiamare `list_products` senza argomenti oppure con `{"category": "Books"}`.

2.8.6 Un'osservazione sulle performance

Ogni invocazione crea un contesto e una connessione nuova. Per SQLite (file locale) il costo è irrisorio. Per database remoti (SQL Server, PostgreSQL) il **connection pooling** del provider rende comunque efficiente questa creazione: la connessione fisica viene riusata da un pool, e il contesto logico è leggero. Il pattern factory, quindi, non è un lusso da prototipo: è la soluzione di produzione.

2.9 I tool di scrittura

Le scritture sono il punto in cui un server MCP diventa potente — e pericoloso. Creazione e cancellazione vanno implementate con cura: validazione, gestione degli errori e annotazioni che informino il client della loro natura.

2.9.1 Creare un prodotto

```
[McpServerTool(Name = "create_product"), Description("Creates a new
↪  product in an existing category.")]
public async Task<string> CreateProduct(
    [Description("Product name.")] string name,
    [Description("Product price.")] decimal price,
    [Description("Category name.")] string categoryName,
    CancellationToken cancellationToken = default)
{
    await using var db = await
↪  factory.CreateDbContextAsync(cancellationToken);

    if (string.IsNullOrWhiteSpace(name))
        throw new McpException("Product name cannot be empty.");

    if (price < 0)
        throw new McpException("Price cannot be negative.");
```

```
    var category = await db.Categories
        .FirstOrDefaultAsync(c => c.Name == categoryName,
        ↪  cancellationToken)
        ?? throw new McpException($"Category not found:
        ↪  {categoryName}");

    var product = new Product { Name = name, Price = price, CategoryId
    ↪  = category.Id };
    db.Products.Add(product);
    await db.SaveChangesAsync(cancellationToken);

    return $"created product '{name}' (id {product.Id})";
}
```

Punti chiave:

- cerchiamo la categoria per nome; se non esiste, lanciamo una `McpException` con un messaggio chiaro — il modello potrà riferirlo all'utente e, eventualmente, riprovare con un'altra categoria;
- creiamo il prodotto con `CategoryId` (la chiave esterna, già nota) invece di navigare la relazione: è la strada più diretta quando abbiamo già caricato la categoria;
- dopo `SaveChangesAsync`, `product.Id` contiene l'identificativo generato dal database: lo restituiamo nel messaggio.

2.9.2 Cancellare un prodotto

```
[McpServerTool(Name = "delete_product", Destructive = true),
 Description("Deletes a product by id. This operation cannot be
  ↪  undone.")]
public async Task<string> DeleteProduct(
    [Description("Product id.")] int id,
    CancellationToken cancellationToken = default)
{
    if (newPrice < 0)
        throw new McpException("Price cannot be negative.");

    await using var db = await
↪   factory.CreateDbContextAsync(cancellationToken);

    var product = await db.Products.FindAsync([id], cancellationToken)
        ?? throw new McpException($"Product not found: {id}");

    db.Products.Remove(product);
    await db.SaveChangesAsync(cancellationToken);

    return $"deleted product {id}";
```

```
}
```

Qui usiamo due accortezze:

1. `FindAsync` cerca per chiave primaria: è più idiomatica di `FirstOrDefaultAsync(p => p.Id == id)` e, con il tracking attivo, può evitare una query se l'entità è già in memoria.
2. `Destructive = true` dichiara che l'operazione è distruttiva: i client possono usare questa annotazione per chiedere conferma all'utente o applicare politiche più prudenti. È un piccolo dettaglio che fa una grande differenza in termini di sicurezza.

2.9.3 Gestire i vincoli del database

Il database ha l'ultima parola. Se, per esempio, tentassimo di creare un prodotto con una categoria inesistente e non avessimo il controllo esplicito, SQLite risponderebbe con un errore di vincolo di chiave esterna. Le eccezioni di EF Core (`DbUpdateException`) incapsulano questi errori: in un server robusto conviene intercettarle e tradurle in messaggi comprensibili.

Un esempio di traduzione:

```
try
{
    await db.SaveChangesAsync(cancellationToken);
}
catch (DbUpdateException)
{
    throw new McpException("Could not save the product: a database
    ↪ constraint was violated.");
}
```

Nella pratica, i controlli espliciti (come la ricerca della categoria prima dell'inserimento) sono preferibili: prevengono l'errore invece di gestirlo.

2.9.4 Un tool di aggiornamento

Per completezza, ecco un aggiornamento del prezzo:

```
[McpServerTool(Name = "update_price"), Description("Updates the price of
↪ an existing product.")]
public async Task<string> UpdatePrice(
    [Description("Product id.")] int id,
    [Description("New price.")] decimal newPrice,
    CancellationToken cancellationToken = default)
{
    await using var db = await
↪   factory.CreateDbContextAsync(cancellationToken);
```

```
    var product = await db.Products.FindAsync([id], cancellationToken)
        ?? throw new McpException($"Product not found: {id}");

    product.Price = newPrice;
    await db.SaveChangesAsync(cancellationToken);

    return $"updated price of product {id} to {newPrice}";
}
```

Qui il **change tracker** fa il lavoro: carichiamo l'entità (che viene tracciata), modifichiamo la proprietà e chiamiamo `SaveChangesAsync`; EF Core genera l'`UPDATE` con le sole colonne cambiate.

2.9.5 Validazione degli input

Gli argomenti arrivano dal modello, che può sbagliare. Oltre ai controlli di esistenza, valida i **vincoli di dominio**:

I controlli vanno applicati nel corpo del tool prima di accedere al database, come negli esempi di `CreateProduct` e `UpdatePrice`.

La regola d'oro è la stessa del Capitolo 1: lo schema dichiara i *tipi*, ma i *vincoli di business* li applichi tu. Un tool che restituisce errori chiari è un tool che il modello impara a usare bene.

2.10 Il pattern IDbContextFactory

Dedichiamo una sezione al pattern che rende solida l'integrazione tra EF Core e MCP. È il punto in cui molti server "fai da te" si rompono, e vale la pena capirlo a fondo.

2.10.1 Il problema del DbContext condiviso

Un `DbContext` non è thread-safe e non è pensato per vivere a lungo. Se registrassimo un singolo contesto e lo condividessimo tra tutte le invocazioni di tool, avremmo:

- **race condition** — due tool che usano lo stesso contesto in parallelo possono corrompere lo stato o lanciare eccezioni;
- **crescita della memoria** — ogni entità caricata resta nel change tracker, consumando memoria e rallentando le query successive;
- **stale data** — il contesto tiene in cache le entità già lette, e rischia di restituire versioni vecchie.

Nelle applicazioni web il problema si risolve con lo scope: un `DbContext` per richiesta. Nei server MCP non c'è una "richiesta" lunga una sola unità di lavoro: c'è una **invocazione di tool** alla volta.

2.10.2 La soluzione: una factory

`AddDbContextFactory<T>` registra una `IDbContextFactory<T>`: un oggetto **singleton e thread-safe** che, a ogni chiamata di `CreateDbContextAsync`, produce un contesto **nuovo e indipendente**. Il pattern d'uso nei tool è sempre lo stesso:

```
await using var db = await
 ↪  factory.CreateDbContextAsync(cancellationToken);
// ... query o scritture ...
```

L'`await using` chiude il contesto al termine del metodo, rilasciando la connessione al pool. Ogni invocazione di tool ha il suo contesto, con il suo tracker e la sua connessione: niente condivisione, niente race condition.

2.10.3 Perché funziona bene con MCP

Il ciclo di vita di un contesto creato dalla factory coincide con il ciclo di vita di un tool:

1. il tool riceve la richiesta;
2. crea il contesto;
3. esegue una o più query;
4. salva le eventuali modifiche;
5. chiude il contesto.

È esattamente l'unità di lavoro ideale per EF Core: corta, isolata, transazionale. E poiché l'SDK MCP **costruisce un'istanza del tool per ogni invocazione** (come abbiamo visto nel Capitolo 1), la factory iniettata nel costruttore è sempre pronta.

2.10.4 Factory vs contesto diretto

Aspetto	`AddDbContext<T>` + iniezione diretta	`AddDbContextFactory<T>`
Lifetime del contesto	Scoped (da gestire)	Per invocazione (esplicito)
Concorrenza	Rischiosa se condiviso	Sicura
Tracking	Accumula	Pulito a ogni chiusura
Adatto a MCP	No	Sì

2.10.5 Un'alternativa valida: il contesto "short-lived" per tool

C'è un altro pattern che funziona: creare il contesto **dentro** il tool con un `DbContextOptionsBuilder` o risolverlo da un provider di servizi. Ma è più

verboso e aggira la dependency injection. La factory è la strada idiomatica, testata e raccomandata: usala.

2.10.6 Cosa ricordare

- Registra con `AddDbContextFactory<T>`, non `AddDbContext<T>`.
- Nei tool, crea il contesto con `await using var db = await factory.CreateDbContextAsync(ct)`.
- Non riusare mai un contesto tra due invocazioni.
- Le connessioni fisiche sono comunque riusate dal pool del provider: non c'è alcun costo nascosto.

Con questo pattern assodato, possiamo assemblare il server completo.

2.11 Il server stdio completo

Mettiamo insieme tutti i pezzi. Il listato che segue è il `Program.cs` completo del nostro server: configurazione, creazione del database con semina, registrazione dei servizi, classe del tool. È il file che puoi compilare ed eseguire così com'è.

2.11.1 Il codice completo

```
using System.ComponentModel;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol;
using ModelContextProtocol.Server;

var builder = Host.CreateApplicationBuilder(args);

// Su stdio lo stdout è del protocollo: niente log su console.
builder.Logging.ClearProviders();

// Registra la factory del DbContext (un contesto nuovo per ogni tool).
builder.Services.AddDbContextFactory<CatalogDbContext>(options =>
    options.UseSqlite("Data Source=catalog.db"));

// Registra il server MCP e il suo tool.
builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithTools<CatalogTool>();

// Crea il database e semina i dati di esempio (solo per l'esempio).
{
    var options = new DbContextOptionsBuilder<CatalogDbContext>()
        .UseSqlite("Data Source=catalog.db").Options;
```

```
    await using var seedDb = new CatalogDbContext(options);
    await seedDb.Database.EnsureCreatedAsync();

    if (!await seedDb.Products.AnyAsync())
    {
        var books = await seedDb.Categories.SingleOrDefaultAsync(c =>
        ↪ c.Name == "Books")
            ?? new Category { Name = "Books" };
        var electronics = await seedDb.Categories.SingleOrDefaultAsync(c
        ↪ => c.Name == "Electronics")
            ?? new Category { Name = "Electronics" };
        seedDb.Products.AddRange(
            new Product { Name = "The Lord of the Rings", Price =
            ↪ 29.90m, Category = books },
            new Product { Name = "MacBook Air", Price = 1199.00m,
            ↪ Category = electronics });
        await seedDb.SaveChangesAsync();
    }
}

await builder.Build().RunAsync();

// ---- Entità ----
public sealed class Category
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public List<Product> Products { get; set; } = [];
}

public sealed class Product
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public decimal Price { get; set; }
    public int CategoryId { get; set; }
    public Category? Category { get; set; }
}

// ---- DbContext ----
public sealed class CatalogDbContext(DbContextOptions<CatalogDbContext>
↪ options)
    : DbContext(options)
{
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Product>().Property(p =>
↪ p.Price).HasPrecision(18, 2);
```

```
        modelBuilder.Entity<Category>().HasIndex(c =>
↪   c.Name).IsUnique();
    }
}

// ---- Tool ----
[McpServerToolType]
public sealed class CatalogTool(IDbContextFactory<CatalogDbContext>
↪   factory)
{
    [McpServerTool(Name = "list_products"), Description("Lists all
    ↪  products, optionally filtered by category name.")]
    public async Task<ProductDto[]> ListProducts(
        [Description("Optional category name filter.")] string? category
        ↪  = null,
        CancellationToken cancellationToken = default)
    {
        await using var db = await
↪   factory.CreateDbContextAsync(cancellationToken);
        IQueryable<Product> query = db.Products;
        if (category is not null)
            query = query.Where(p => p.Category!.Name == category);
        return await query.OrderBy(p => p.Name)
            .Select(p => new ProductDto(p.Id, p.Name, p.Price,
            ↪  p.Category!.Name))
            .ToArrayAsync(cancellationToken);
    }

    [McpServerTool(Name = "search_products"), Description("Searches
    ↪  products whose name contains the given text.")]
    public async Task<ProductDto[]> SearchProducts(
        [Description("Text to search in product names.")] string term,
        CancellationToken cancellationToken = default)
    {
        await using var db = await
↪   factory.CreateDbContextAsync(cancellationToken);
        return await db.Products
            .Where(p => p.Name.Contains(term))
            .OrderBy(p => p.Name)
            .Select(p => new ProductDto(p.Id, p.Name, p.Price,
            ↪  p.Category!.Name))
            .ToArrayAsync(cancellationToken);
    }

    [McpServerTool(Name = "get_product"), Description("Gets a single
    ↪  product by id.")]
    public async Task<ProductDto?> GetProduct(
        [Description("Product id.")] int id,
        CancellationToken cancellationToken = default)
    {
```

```
        await using var db = await
↪   factory.CreateDbContextAsync(cancellationToken);
        return await db.Products
            .Where(p => p.Id == id)
            .Select(p => new ProductDto(p.Id, p.Name, p.Price,
            ↪  p.Category!.Name))
            .FirstOrDefaultAsync(cancellationToken);
    }

    [McpServerTool(Name = "create_product"), Description("Creates a new
    ↪  product in an existing category.")]
    public async Task<string> CreateProduct(
        [Description("Product name.")] string name,
        [Description("Product price.")] decimal price,
        [Description("Category name.")] string categoryName,
        CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrWhiteSpace(name))
            throw new McpException("Product name cannot be empty.");
        if (price < 0)
            throw new McpException("Price cannot be negative.");

        await using var db = await
↪   factory.CreateDbContextAsync(cancellationToken);
        var category = await db.Categories.FirstOrDefaultAsync(c =>
        ↪  c.Name == categoryName, cancellationToken)
            ?? throw new McpException($"Category not found:
            ↪  {categoryName}");
        var product = new Product { Name = name, Price = price,
        ↪  CategoryId = category.Id };
        db.Products.Add(product);
        await db.SaveChangesAsync(cancellationToken);
        return $"created product '{name}' (id {product.Id})";
    }

    [McpServerTool(Name = "update_price"), Description("Updates the
    ↪  price of an existing product.")]
    public async Task<string> UpdatePrice(
        [Description("Product id.")] int id,
        [Description("New price.")] decimal newPrice,
        CancellationToken cancellationToken = default)
    {
        if (newPrice < 0)
            throw new McpException("Price cannot be negative.");

        await using var db = await
↪   factory.CreateDbContextAsync(cancellationToken);
        var product = await db.Products.FindAsync([id],
        ↪  cancellationToken)
            ?? throw new McpException($"Product not found: {id}");
```

```
        product.Price = newPrice;
        await db.SaveChangesAsync(cancellationToken);
        return $"updated price of product {id} to {newPrice}";
    }

    [McpServerTool(Name = "delete_product", Destructive = true),
     Description("Deletes a product by id. This operation cannot be
      ↪  undone.")]
    public async Task<string> DeleteProduct(
        [Description("Product id.")] int id,
        CancellationToken cancellationToken = default)
    {
        await using var db = await
 ↪  factory.CreateDbContextAsync(cancellationToken);
        var product = await db.Products.FindAsync([id],
         ↪  cancellationToken)
            ?? throw new McpException($"Product not found: {id}");
        db.Products.Remove(product);
        await db.SaveChangesAsync(cancellationToken);
        return $"deleted product {id}";
    }
}

public sealed record ProductDto(int Id, string Name, decimal Price,
 ↪  string Category);
```

2.11.2 Eseguire e provare

Avviamo il server:

```
> dotnet run
```

E collaudiamolo con una sessione JSON-RPC manuale oppure, meglio, con l'MCP Inspector. Un esempio di chiamata:

```
{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{
  "name":"list_products",
  "arguments":{"category":"Books"}
}}
```

Il server risponderà con l'elenco dei prodotti della categoria "Books", proiettati nel DTO.

2.11.3 Osservazioni sul codice completo

- Questo listato è una **demo non destinata alla produzione**: `EnsureCreated` e la semina idempotente preparano il database se necessario, senza cancellarlo a ogni avvio. Per usare migrazioni, rimuovi l'inizializzazione dimostrativa e applicale con `dotnet ef`; non inserire `EnsureDe-`

leted nel codice di avvio, perché EF Core può eseguirlo anche in fase di design.
- La classe del tool espone sei strumenti: elenco, ricerca, lettura singola, creazione, aggiornamento del prezzo e cancellazione. Mantieni in `Name` lo stesso nome snake_case atteso dal client.
- Tutto il codice è stato compilato ed eseguito con .NET 10 e i pacchetti indicati: il round-trip completo (creazione database, semina, `list_products`, `create_product`, `delete_product`) è stato verificato.

2.11.4 Separare i file

In un progetto reale, `Program.cs` non dovrebbe contenere tutto. Una struttura pulita è:

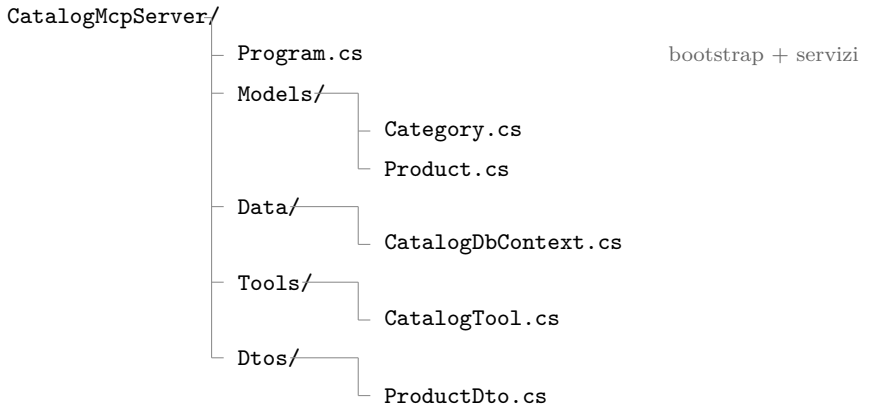


Figura 2: Struttura consigliata del progetto `CatalogMcpServer`

La logica non cambia: cambia solo l'organizzazione. Nel libro teniamo tutto insieme per leggibilità, ma nei tuoi progetti separa per responsabilità.

2.12 Il server HTTP completo

Se il catalogo deve essere condiviso tra più utenti o macchine, il trasporto `stdio` non basta: serve HTTP. Come nel Capitolo 1, il passaggio è questione di pochi ritocchi.

2.12.1 Il pacchetto aggiuntivo

```
> dotnet add package ModelContextProtocol.AspNetCore --version 2.3.0
```

2.12.2 Le differenze rispetto a stdio

Tre modifiche al codice precedente:

1. `WebApplication.CreateBuilder` al posto di `Host.CreateApplicationBuilder`;
2. `WithHttpTransport()` al posto di `WithStdioServerTransport()`;
3. `app.MapMcp("/mcp")` per mappare l'endpoint sullo stesso percorso usato dai client.

Tutto il resto — entità, `DbContext`, tool — resta **identico** alla sezione 2.11: i metodi dei tool dichiarano esplicitamente il nome pubblico, per esempio `[McpServerTool(Name = "list_products")]`. È la riprova del valore del protocollo: il codice di dominio non dipende dal trasporto.

```
using System.ComponentModel;
using Microsoft.AspNetCore.Builder;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using ModelContextProtocol;
using ModelContextProtocol.Server;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContextFactory<CatalogDbContext>(options =>
    options.UseSqlite("Data Source=catalog.db"));

builder.Services.AddMcpServer()
    .WithHttpTransport()
    .WithTools<CatalogTool>();

var app = builder.Build();

// Inizializzazione dimostrativa, come nella versione stdio; non
↪  cancella il database.
{
    var options = new DbContextOptionsBuilder<CatalogDbContext>()
        .UseSqlite("Data Source=catalog.db").Options;
    await using var seedDb = new CatalogDbContext(options);
    await seedDb.Database.EnsureCreatedAsync();

    if (!await seedDb.Products.AnyAsync())
    {
        var books = await seedDb.Categories.SingleOrDefaultAsync(c =>
        ↪  c.Name == "Books")
            ?? new Category { Name = "Books" };
        var electronics = await seedDb.Categories.SingleOrDefaultAsync(c
        ↪  => c.Name == "Electronics")
            ?? new Category { Name = "Electronics" };
        seedDb.Products.AddRange(
```

```
            new Product { Name = "The Lord of the Rings", Price =
             ↪  29.90m, Category = books },
            new Product { Name = "MacBook Air", Price = 1199.00m,
             ↪  Category = electronics });
        await seedDb.SaveChangesAsync();
    }
}

app.MapMcp("/mcp");

await app.RunAsync();

// ... entità, DbContext e CatalogTool identici alla versione stdio ...
```

2.12.3 Avviare e collegare

Avviamo il server su una porta:

```
> dotnet run --urls http://localhost:5080
```

Il client (Claude Desktop, VS Code o l'Inspector) userà l'URL `http://localhost:5080/mcp`. La configurazione per Claude Desktop, per esempio:

```
{
  "mcpServers": {
    "catalog": {
      "url": "http://localhost:5080/mcp"
    }
  }
}
```

2.12.4 Considerazioni per il database in HTTP

Come il listato `stdio`, anche questo è un esempio didattico, non una configurazione di produzione. `EnsureCreated` non sostituisce le migrazioni: per usare `dotnet ef`, rimuovi il blocco dimostrativo di inizializzazione e applica le migrazioni separatamente. Non aggiungere `EnsureDeleted` al codice eseguito all'avvio, perché può essere eseguito anche quando gli strumenti EF Core caricano l'app in fase di design.

Con un server HTTP condiviso, il database diventa una **risorsa concorrente**: più richieste possono arrivare contemporaneamente. Qui il pattern `IDbContextFactory` dà il meglio di sé: ogni richiesta crea il proprio contesto, e SQLite (o il provider scelto) gestisce la concorrenza a livello di file o di server. Due accortezze:

- per SQLite, in scenari ad alta concorrenza di scrittura, valuta di abilitare il **WAL** (Write-Ahead Logging) per ridurre i blocchi;
- per database server (SQL Server, PostgreSQL), la concorrenza è gestita dal motore, e il pooling delle connessioni rende tutto fluido.

2.12.5 Un consiglio di architettura

Se il catalogo deve servire sia un'applicazione tradizionale sia un agente AI, non duplicare la logica: costruisci le tue **REST API** per l'applicazione e un **MCP server** che usa lo stesso `DbContext` (o gli stessi servizi applicativi) per l'agente. Come discusso nel Capitolo 1, REST e MCP sono complementari, e condividere il livello dati è la scelta che massimizza la coerenza e minimizza la manutenzione.

2.13 Testare il server con l'MCP Inspector

Come per il Capitolo 1, l'MCP Inspector è il banco di prova ideale. Per un server con database, però, il collaudo ha un'attenzione in più: verificare che lo **stato** cambi correttamente tra una chiamata e l'altra.

2.13.1 Avviare l'Inspector

```
> npx @modelcontextprotocol/inspector dotnet run --project
 ↪  C:\percorso\CatalogMcpServer
```

Per la versione HTTP:

```
> npx @modelcontextprotocol/inspector --transport http
 ↪  http://localhost:5080/mcp
```

2.13.2 La sequenza di collaudo

Esegui queste operazioni in ordine e verifica i risultati:

1. **`list_products` senza argomenti** — deve restituire i due prodotti seminati all'avvio.
2. **`list_products` con `{"category": "Books"}`** — deve restituire solo il prodotto della categoria "Books".
3. **`search_products` con `{"term": "Mac"}`** — deve trovare "MacBook Air".
4. **`create_product`** con `{"name": "Headphones", "price": 199.99, "categoryName": "Electronics"}` — deve creare il prodotto e restituire l'id.
5. **`list_products` di nuovo** — deve ora mostrare il nuovo prodotto: questo conferma che il **database persiste** le modifiche.
6. **`delete_product`** con l'id del prodotto appena creato — deve eliminarlo.

7. **`create_product` con una categoria inesistente** — deve restituire l'errore `Category not found: ...`, non un crash.
8. **`delete_product` con un id inesistente** — deve restituire `Product not found: ...`.

2.13.3 Cosa osservare negli errori

Una `McpException` lanciata dal tool non è un errore di protocollo JSON-RPC: `tools/call` restituisce normalmente un risultato con `isError: true` e il messaggio in `content`, per esempio:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [{ "type": "text", "text": "Product not found: 999" }],
    "isError": true
  }
}
```

Il campo JSON-RPC di primo livello `error` è invece riservato agli errori di protocollo, per esempio una richiesta non valida o un metodo sconosciuto. Verifica che il risultato del tool sia **chiaro e contestualizzato**, non un generico "internal error": se il modello riceve un errore chiaro, può correggersi; se riceve un errore opaco, fallisce e frustra l'utente.

2.13.4 Testare la persistenza attraverso il riavvio

Un test che vale la pena fare: crea un prodotto, **chiudi l'Inspector** (che termina il server), riavvia il server e verifica con `list_products` che il prodotto esista ancora. L'inizializzazione dimostrativa usa `EnsureCreated` senza cancellare il database e semina i prodotti solo se non ce ne sono: il prodotto creato deve quindi persistere al riavvio. È comunque un artificio didattico, non una strategia di inizializzazione per la produzione.

2.13.5 Test automatici con un client .NET

Nel Capitolo 1 abbiamo visto come scrivere un client .NET. Lo stesso approccio si applica qui e diventa un **test di integrazione** prezioso: avvia il server, invoca `create_product`, poi `list_products`, e verifica che il nuovo prodotto compaia. Il pattern è identico a quello già mostrato; la differenza è che ora, dietro i tool, c'è un database reale.

2.14 Primitivi avanzati: il Progress

Finora i nostri tool sono "tutto o niente": partono, lavorano e restituiscono un risultato. Ma un'operazione lunga — per esempio una query pesante, un'impor-

tazione o un calcolo — trae beneficio dal **reporting del progresso**: il server informa il client di quanto manca al completamento, e il client può mostrarlo all'utente.

2.14.1 Il meccanismo

MCP prevede che il client, quando invoca un tool, possa allegare un **progress token**. Il server, durante l'esecuzione, invia notifiche `notifications/progress` con quel token, comunicando una frazione di avanzamento (0..1), un totale opzionale e un messaggio.

L'SDK .NET rende tutto questo trasparente: basta dichiarare nel tool un parametro di tipo `IProgress<ProgressNotificationValue>`. L'SDK lo riconosce come parametro speciale, lo **esclude dallo schema di input** e lo inietta automaticamente, collegato al progress token del client.

2.14.2 Un tool con progress

Estendiamo il nostro catalogo con un tool che conta i prodotti simulando un lavoro lungo:

```
using ModelContextProtocol.Protocol;

[McpServerTool, Description("Counts products, reporting progress while
 ↪  working.")]
public async Task<string> CountWithProgress(
    IProgress<ProgressNotificationValue>? progress = null,
    CancellationToken cancellationToken = default)
{
    await using var db = await
 ↪  factory.CreateDbContextAsync(cancellationToken);

    progress?.Report(new ProgressNotificationValue
    {
        Progress = 0.2f,
        Total = 1f,
        Message = "querying the database"
    });

    var total = await db.Products.CountAsync(cancellationToken);

    progress?.Report(new ProgressNotificationValue
    {
        Progress = 1f,
        Total = 1f,
        Message = "done"
    });

    return $"counted {total} products";
}
```

Osservazioni:

- `Progress` e `Total` sono di tipo `float`: da qui i suffissi `f` sui letterali.
- Il parametro è opzionale (`? progress = null`): se il client non fornisce un progress token, l'SDK passa `null` e il tool funziona comunque.
- Lo schema di input del tool resta **vuoto** (`{"type":"object","properties":{}}`): `IProgress` non è un argomento, è un canale.

2.14.3 Perché è utile

In un tool che importa migliaia di righe o interroga più sistemi, il progress trasforma un'attesa cieca in un'esperienza informata: l'host può mostrare una barra o un messaggio, e il modello può decidere se attendere o suggerire alternative. È un primitivo semplice ma che aumenta molto la percezione di affidabilità di un agente.

2.14.4 Una nota sul token

Non devi gestire il token a mano: l'SDK associa il parametro `IProgress` al token della richiesta corrente. Se più invocazioni avvengono in parallelo, ogni tool ha il suo canale di progress isolato. È l'ennesimo caso in cui l'SDK trasforma una complessità di protocollo in un semplice costrutto C#.

2.15 Riepilogo, glossario ed esercizi

Abbiamo trasformato il catalogo MCP senza stato del Capitolo 1 in un servizio che consulta e modifica dati persistenti. Ripercorriamo le scelte principali e usiamole come base per la pratica.

2.15.1 Riepilogo

- Entity Framework Core traduce le query LINQ in operazioni sul database; `Select` definisce i dati restituiti e spesso rende superfluo `Include`.
- Un `DbContext` non va condiviso tra invocazioni concorrenti: `IDbContext-Factory<TContext>` permette di creare un contesto per ogni chiamata del tool.
- I DTO limitano i dati esposti al modello e separano il contratto MCP dalle entità persistite.
- Gli input del modello vanno convalidati come qualsiasi altro input esterno; le scritture e le operazioni distruttive richiedono particolare cautela.
- `EnsureCreated` e la semina idempotente sono comodi per una demo, ma non sostituiscono le migrazioni né una strategia di deployment.

- `stdio` e Streamable HTTP cambiano il trasporto, non il contratto dei tool; Progress comunica l'avanzamento quando un'operazione richiede tempo.

2.15.2 Glossario

Termine	Significato
ORM	Libreria che mappa oggetti e relazioni alle strutture di un database.
DbContext	Unità di lavoro EF Core che gestisce query, tracking e salvataggi.
DTO	Oggetto che definisce i dati scambiati attraverso un contratto esterno.
Migration	Modifica versionata dello schema del database, applicabile in modo controllato.
Progress token	Token fornito dal client per correlare le notifiche di avanzamento di una richiesta.

2.15.3 Esercizi proposti

1. Estendi `list_products` con un intervallo di prezzo opzionale e verifica che il filtro venga eseguito dal database.
2. Scrivi un test d'integrazione per `delete_product` e verifica che il client possa rilevare l'annotazione distruttiva.
3. Scrivi un test d'integrazione per la ricerca per nome e specifica il comportamento atteso quando il termine è vuoto.
4. Aggiungi una migration e prova ad applicarla a un database di sviluppo senza cancellare i dati esistenti.
5. Estendi il tool con Progress per un'importazione simulata; gestisci la cancellazione e comunica solo avanzamenti significativi.